# Báo cáo bài tập thực hành tuần 1

> Họ và tên: Nguyễn Vạn Phúc Huy
> MSSV: 23110163
> Lớp: 23TTH (Chiều thứ 6, ca 1)

Import các thư viện cần thiết

In [20]:
```
import numpy as np
import pandas as pd
import scipy
from sklearn.preprocessing import StandardScaler, KBinsDiscretizer, MinMaxScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
```

Note: Câu 1 không giải thích nhiều vì làm lại y chang trong file pdf

## 1. Làm sạch và tiền xử lý dữ liệu:

Như yêu cầu nên sẽ không giải thích chi tiết các bước làm sạch và tiền xử lý dữ liệu, chỉ trình bày code thực hiện.

## 2. Bài tập vận dụng

Đọc dữ liệu trực tiếp link đã cung cấp thay vì tải về vì em lười ...

In [32]:
```
link_hd_ko_che= r'https://archive.ics.uci.edu/ml/machine-learning-databases/arrhythmia/arrhythmi
df = pd.read_csv(link_hd_ko_che, header=None)
df
```

Out[32]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... | 270 | 271 | 272 | 273 | 274 | 275 | 276 | 277 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 75 | 0 | 190 | 80 | 91 | 193 | 371 | 174 | 121 | -16 | ... | 0.0 | 9.0 | -0.9 | 0.0 | 0.0 | 0.9 | 2.9 | 23.3 | 49 |
| **1** | 56 | 1 | 165 | 64 | 81 | 174 | 401 | 149 | 39 | 25 | ... | 0.0 | 8.5 | 0.0 | 0.0 | 0.0 | 0.2 | 2.1 | 20.4 | 38 |
| **2** | 54 | 0 | 172 | 95 | 138 | 163 | 386 | 185 | 102 | 96 | ... | 0.0 | 9.5 | -2.4 | 0.0 | 0.0 | 0.3 | 3.4 | 12.3 | 49 |
| **3** | 55 | 0 | 175 | 94 | 100 | 202 | 380 | 179 | 143 | 28 | ... | 0.0 | 12.2 | -2.2 | 0.0 | 0.0 | 0.4 | 2.6 | 34.6 | 61 |
| **4** | 75 | 0 | 190 | 80 | 88 | 181 | 360 | 177 | 103 | -16 | ... | 0.0 | 13.1 | -3.6 | 0.0 | 0.0 | -0.1 | 3.9 | 25.4 | 62 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **447** | 53 | 1 | 160 | 70 | 80 | 199 | 382 | 154 | 117 | -37 | ... | 0.0 | 4.3 | -5.0 | 0.0 | 0.0 | 0.7 | 0.6 | -4.4 | -( |
| **448** | 37 | 0 | 190 | 85 | 100 | 137 | 361 | 201 | 73 | 86 | ... | 0.0 | 15.6 | -1.6 | 0.0 | 0.0 | 0.4 | 2.4 | 38.0 | 62 |
| **449** | 36 | 0 | 166 | 68 | 108 | 176 | 365 | 194 | 116 | -85 | ... | 0.0 | 16.3 | -28.6 | 0.0 | 0.0 | 1.5 | 1.0 | -44.2 | -33 |
| **450** | 32 | 1 | 155 | 55 | 93 | 106 | 386 | 218 | 63 | 54 | ... | -0.4 | 12.0 | -0.7 | 0.0 | 0.0 | 0.5 | 2.4 | 25.0 | 46 |
| **451** | 78 | 1 | 160 | 70 | 79 | 127 | 364 | 138 | 78 | 28 | ... | 0.0 | 10.4 | -1.8 | 0.0 | 0.0 | 0.5 | 1.6 | 21.3 | 32 |

452 rows × 280 columns

Hiện thị thông tin tổng quan bộ dữ liệu

In [33]:
```
df.info()
df.describe()
```

```
print(df.dtypes)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 452 entries, 0 to 451
Columns: 280 entries, 0 to 279
dtypes: float64(120), int64(155), object(5)
memory usage: 988.9+ KB
0        int64
1        int64
2        int64
3        int64
4        int64
        ...
275    float64
276    float64
277    float64
278    float64
279      int64
Length: 280, dtype: object
```

Ta thực hiện làm sạch dữ liệu bằng cách loại bỏ các dòng có giá trị thiếu và các dòng có giá trị trùng lặp (nếu có)

In [34]:
```
print(df.isnull().sum())
df = df.dropna()

df
```

```
0      0
1      0
2      0
3      0
4      0
      ..
275    0
276    0
277    0
278    0
279    0
Length: 280, dtype: int64
```

Out[34]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... | 270 | 271 | 272 | 273 | 274 | 275 | 276 | 277 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 75 | 0 | 190 | 80 | 91 | 193 | 371 | 174 | 121 | -16 | ... | 0.0 | 9.0 | -0.9 | 0.0 | 0.0 | 0.9 | 2.9 | 23.3 | 49 |
| **1** | 56 | 1 | 165 | 64 | 81 | 174 | 401 | 149 | 39 | 25 | ... | 0.0 | 8.5 | 0.0 | 0.0 | 0.0 | 0.2 | 2.1 | 20.4 | 38 |
| **2** | 54 | 0 | 172 | 95 | 138 | 163 | 386 | 185 | 102 | 96 | ... | 0.0 | 9.5 | -2.4 | 0.0 | 0.0 | 0.3 | 3.4 | 12.3 | 49 |
| **3** | 55 | 0 | 175 | 94 | 100 | 202 | 380 | 179 | 143 | 28 | ... | 0.0 | 12.2 | -2.2 | 0.0 | 0.0 | 0.4 | 2.6 | 34.6 | 61 |
| **4** | 75 | 0 | 190 | 80 | 88 | 181 | 360 | 177 | 103 | -16 | ... | 0.0 | 13.1 | -3.6 | 0.0 | 0.0 | -0.1 | 3.9 | 25.4 | 62 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **447** | 53 | 1 | 160 | 70 | 80 | 199 | 382 | 154 | 117 | -37 | ... | 0.0 | 4.3 | -5.0 | 0.0 | 0.0 | 0.7 | 0.6 | -4.4 | -( |
| **448** | 37 | 0 | 190 | 85 | 100 | 137 | 361 | 201 | 73 | 86 | ... | 0.0 | 15.6 | -1.6 | 0.0 | 0.0 | 0.4 | 2.4 | 38.0 | 62 |
| **449** | 36 | 0 | 166 | 68 | 108 | 176 | 365 | 194 | 116 | -85 | ... | 0.0 | 16.3 | -28.6 | 0.0 | 0.0 | 1.5 | 1.0 | -44.2 | -33 |
| **450** | 32 | 1 | 155 | 55 | 93 | 106 | 386 | 218 | 63 | 54 | ... | -0.4 | 12.0 | -0.7 | 0.0 | 0.0 | 0.5 | 2.4 | 25.0 | 46 |
| **451** | 78 | 1 | 160 | 70 | 79 | 127 | 364 | 138 | 78 | 28 | ... | 0.0 | 10.4 | -1.8 | 0.0 | 0.0 | 0.5 | 1.6 | 21.3 | 32 |

452 rows × 280 columns

In [35]:
```
print(df.duplicated().sum())
df = df.drop_duplicates()
df
```

0

Out[35]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... | 270 | 271 | 272 | 273 | 274 | 275 | 276 | 277 | 2 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 75 | 0 | 190 | 80 | 91 | 193 | 371 | 174 | 121 | -16 | ... | 0.0 | 9.0 | -0.9 | 0.0 | 0.0 | 0.9 | 2.9 | 23.3 | 49 |
| **1** | 56 | 1 | 165 | 64 | 81 | 174 | 401 | 149 | 39 | 25 | ... | 0.0 | 8.5 | 0.0 | 0.0 | 0.0 | 0.2 | 2.1 | 20.4 | 38 |
| **2** | 54 | 0 | 172 | 95 | 138 | 163 | 386 | 185 | 102 | 96 | ... | 0.0 | 9.5 | -2.4 | 0.0 | 0.0 | 0.3 | 3.4 | 12.3 | 49 |
| **3** | 55 | 0 | 175 | 94 | 100 | 202 | 380 | 179 | 143 | 28 | ... | 0.0 | 12.2 | -2.2 | 0.0 | 0.0 | 0.4 | 2.6 | 34.6 | 61 |
| **4** | 75 | 0 | 190 | 80 | 88 | 181 | 360 | 177 | 103 | -16 | ... | 0.0 | 13.1 | -3.6 | 0.0 | 0.0 | -0.1 | 3.9 | 25.4 | 62 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **447** | 53 | 1 | 160 | 70 | 80 | 199 | 382 | 154 | 117 | -37 | ... | 0.0 | 4.3 | -5.0 | 0.0 | 0.0 | 0.7 | 0.6 | -4.4 | -( |
| **448** | 37 | 0 | 190 | 85 | 100 | 137 | 361 | 201 | 73 | 86 | ... | 0.0 | 15.6 | -1.6 | 0.0 | 0.0 | 0.4 | 2.4 | 38.0 | 62 |
| **449** | 36 | 0 | 166 | 68 | 108 | 176 | 365 | 194 | 116 | -85 | ... | 0.0 | 16.3 | -28.6 | 0.0 | 0.0 | 1.5 | 1.0 | -44.2 | -33 |
| **450** | 32 | 1 | 155 | 55 | 93 | 106 | 386 | 218 | 63 | 54 | ... | -0.4 | 12.0 | -0.7 | 0.0 | 0.0 | 0.5 | 2.4 | 25.0 | 46 |
| **451** | 78 | 1 | 160 | 70 | 79 | 127 | 364 | 138 | 78 | 28 | ... | 0.0 | 10.4 | -1.8 | 0.0 | 0.0 | 0.5 | 1.6 | 21.3 | 32 |

452 rows × 280 columns

Tiếp theo thì ta scaler dữ liệu để chuẩn hóa dữ liệu về cùng một thang đo

In [36]:

```
scaler = StandardScaler()
array = df.values

robust_array = scaler.fit_transform(array)
robust_df = pd.DataFrame(robust_array, columns=df.columns)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[36], line 4
      1 scaler = StandardScaler()
      2 array = df.values
----> 4 robust_array = scaler.fit_transform(array)
      5 robust_df = pd.DataFrame(robust_array, columns=df.columns)

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\_set
_output.py:316, in _wrap_method_output.<locals>.wrapped(self, X, *args, **kwargs)
    314 @wraps(f)
    315 def wrapped(self, X, *args, **kwargs):
--> 316     data_to_wrap = f(self, X, *args, **kwargs)
    317     if isinstance(data_to_wrap, tuple):
    318         # only wrap the first output for cross decomposition
    319         return_tuple = (
    320             _wrap_data_with_container(method, data_to_wrap[0], X, self),
    321             *data_to_wrap[1:],
    322         )

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\base.py:90
7, in TransformerMixin.fit_transform(self, X, y, **fit_params)
    892         warnings.warn(
    893             (
    894                 f"This object ({self.__class__.__name__}) has a `transform`"
   (...)    902             UserWarning,
    903         )
    905 if y is None:
    906     # fit method of arity 1 (unsupervised transformation)
--> 907     return self.fit(X, **fit_params).transform(X)
    908 else:
    909     # fit method of arity 2 (supervised transformation)
    910     return self.fit(X, y, **fit_params).transform(X)

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocess
ing\_data.py:924, in StandardScaler.fit(self, X, y, sample_weight)
    922 # Reset internal state before fitting
    923 self._reset()
--> 924 return self.partial_fit(X, y, sample_weight)

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\base.py:13
36, in _fit_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
   1329     estimator._validate_params()
   1331 with config_context(
   1332     skip_parameter_validation=(
   1333         prefer_skip_nested_validation or global_skip_validation
   1334     )
   1335 ):
-> 1336     return fit_method(estimator, *args, **kwargs)

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocess
ing\_data.py:961, in StandardScaler.partial_fit(self, X, y, sample_weight)
    959 xp, _, X_device = get_namespace_and_device(X)
    960 first_call = not hasattr(self, "n_samples_seen_")
--> 961 X = validate_data(
    962     self,
    963     X,
    964     accept_sparse=(      ,       ),
    965     dtype=supported_float_dtypes(xp, X_device),
    966     ensure_all_finite=            ,
    967     reset=first_call,
    968 )
    969 n_features = X.shape[1]
    971 if sample_weight is not None:

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\vali
dation.py:2902, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array,
**check_params)
   2900         out = X, y
```

```
   2901 elif not no_val_X and no_val_y:
-> 2902     out = check_array(X, input_name=    , **check_params)
   2903 elif no_val_X and not no_val_y:
   2904     out = _check_y(y, **check_params)

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\vali
dation.py:1022, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, for
ce_writeable, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, en
sure_min_features, estimator, input_name)
   1020         array = xp.astype(array, dtype, copy=False)
   1021     else:
-> 1022         array = _asarray_with_order(array, order=order, dtype=dtype, xp=xp)
   1023 except ComplexWarning as complex_warning:
   1024     raise ValueError(
   1025         "Complex data not supported\n{}\n".format(array)
   1026     ) from complex_warning

File c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\utils\_arr
ay_api.py:878, in _asarray_with_order(array, dtype, order, copy, xp, device)
    876     array = numpy.array(array, order=order, dtype=dtype)
    877 else:
--> 878     array = numpy.asarray(array, order=order, dtype=dtype)
    880 # At this point array is a NumPy ndarray. We convert it to an array
    881 # container that is consistent with the input's namespace.
    882 return xp.asarray(array)

ValueError: could not convert string to float: '?'
```

Ta thấy dòng code ở trên bị lỗi không chạy với error như sau

```
ValueError: could not convert string to float: '?'
```

Nên ta thực hiện lọc '?'

In [ ]:
```
(df== '?').any().value_counts()

df.replace('?', 0, inplace=True)
```

Loại bỏ các outliers

In [ ]:
```
scaler = StandardScaler()
array = df.values

robust_array = scaler.fit_transform(array)
robust_df = pd.DataFrame(robust_array, columns=df.columns)

print('Loai bo cac outliers: \n', robust_df)
```

```
Loai bo cac outliers:
           0         1         2         3         4         5         6    \
0    1.734439 -1.107520  0.641327  0.713814  0.135505  0.844945  0.113709
1    0.579312  0.902918 -0.031998 -0.251644 -0.516072  0.420769  1.013301
2    0.457720 -1.107520  0.156533  1.618932  3.197915  0.175193  0.563505
3    0.518516 -1.107520  0.237332  1.558590  0.721924  1.045871  0.383587
4    1.734439 -1.107520  0.641327  0.713814 -0.059968  0.577044 -0.216141
..        ...       ...       ...       ...       ...       ...       ...
447  0.396924  0.902918 -0.166663  0.110403 -0.581229  0.978896  0.443560
448 -0.575815 -1.107520  0.641327  1.015520  0.721924 -0.405260 -0.186154
449 -0.636611 -1.107520 -0.005065 -0.010279  1.243185  0.465419 -0.066209
450 -0.879796  0.902918 -0.301328 -0.794714  0.265820 -1.097337  0.563505
451  1.916828  0.902918 -0.166663  0.110403 -0.646387 -0.628511 -0.096195

          7         8         9    ...       270       271        272  \
0    0.113809  1.201469 -1.094661  ...  0.508843 -0.013839   0.278621
1   -0.588564 -1.977064 -0.191203  ...  0.508843 -0.157972   0.728573
2    0.422853  0.464980  1.373324  ...  0.508843  0.130294  -0.471299
3    0.254284  2.054247 -0.125096  ...  0.508843  0.908612  -0.371310
4    0.198094  0.503742 -1.094661  ...  0.508843  1.168051  -1.071235
..        ...       ...       ...  ...       ...       ...        ...
447 -0.448089  1.046419 -1.557409  ...  0.508843 -1.368689  -1.771161
448  0.872372 -0.659136  1.152968  ...  0.508843  1.888716  -0.071342
449  0.675708  1.007656 -2.615116  ...  0.508843  2.090502 -13.569901
450  1.349985 -1.046762  0.447829  ... -0.220727  0.850959   0.378610
451 -0.897608 -0.465323 -0.125096  ...  0.508843  0.389733  -0.171331

          273  274       275       276       277       278       279
0   -0.079546  0.0  1.109553  1.177737  0.294603  1.078670  0.935771
1   -0.079546  0.0 -0.906889  0.616126  0.079613  0.504874  0.481455
2   -0.079546  0.0 -0.618826  1.528744 -0.520878  1.057018  1.390087
3   -0.079546  0.0 -0.330763  0.967133  1.132324  1.739077 -0.654336
4   -0.079546  0.0 -1.771079  1.879751  0.450286  1.804035  0.708613
..        ...  ...       ...       ...       ...       ...       ...
447 -0.079546  0.0  0.533427 -0.436895 -1.758926 -1.622502 -0.654336
448 -0.079546  0.0 -0.330763  0.826730  1.384382  1.782383  1.390087
449 -0.079546  0.0  2.837932 -0.156089 -4.709483 -3.392610 -0.427178
450 -0.079546  0.0 -0.042700  0.826730  0.420632  0.927101 -0.654336
451 -0.079546  0.0 -0.042700  0.265119  0.146334  0.180084 -0.654336

[452 rows x 280 columns]
```

Chuẩn hóa và scaling dữ liệu bằng pp z-score và pp min-max

In [ ]:
```
# phuong pháp StandardScaler
scaler = StandardScaler()
array = df.values

standard = scaler.fit_transform(array)
standard_df = pd.DataFrame(standard, columns=df.columns)
print('Chuan hoa du lieu:\n', standard_df)
```

```
Chuan hoa du lieu:
           0         1         2         3         4         5         6    \
0    1.734439 -1.107520  0.641327  0.713814  0.135505  0.844945  0.113709
1    0.579312  0.902918 -0.031998 -0.251644 -0.516072  0.420769  1.013301
2    0.457720 -1.107520  0.156533  1.618932  3.197915  0.175193  0.563505
3    0.518516 -1.107520  0.237332  1.558590  0.721924  1.045871  0.383587
4    1.734439 -1.107520  0.641327  0.713814 -0.059968  0.577044 -0.216141
..        ...       ...       ...       ...       ...       ...       ...
447  0.396924  0.902918 -0.166663  0.110403 -0.581229  0.978896  0.443560
448 -0.575815 -1.107520  0.641327  1.015520  0.721924 -0.405260 -0.186154
449 -0.636611 -1.107520 -0.005065 -0.010279  1.243185  0.465419 -0.066209
450 -0.879796  0.902918 -0.301328 -0.794714  0.265820 -1.097337  0.563505
451  1.916828  0.902918 -0.166663  0.110403 -0.646387 -0.628511 -0.096195

          7         8         9    ...       270       271        272  \
0    0.113809  1.201469 -1.094661  ...  0.508843 -0.013839   0.278621
1   -0.588564 -1.977064 -0.191203  ...  0.508843 -0.157972   0.728573
2    0.422853  0.464980  1.373324  ...  0.508843  0.130294  -0.471299
3    0.254284  2.054247 -0.125096  ...  0.508843  0.908612  -0.371310
4    0.198094  0.503742 -1.094661  ...  0.508843  1.168051  -1.071235
..        ...       ...       ...  ...       ...       ...        ...
447 -0.448089  1.046419 -1.557409  ...  0.508843 -1.368689  -1.771161
448  0.872372 -0.659136  1.152968  ...  0.508843  1.888716  -0.071342
449  0.675708  1.007656 -2.615116  ...  0.508843  2.090502 -13.569901
450  1.349985 -1.046762  0.447829  ... -0.220727  0.850959   0.378610
451 -0.897608 -0.465323 -0.125096  ...  0.508843  0.389733  -0.171331

          273  274       275       276       277       278       279
0   -0.079546  0.0  1.109553  1.177737  0.294603  1.078670  0.935771
1   -0.079546  0.0 -0.906889  0.616126  0.079613  0.504874  0.481455
2   -0.079546  0.0 -0.618826  1.528744 -0.520878  1.057018  1.390087
3   -0.079546  0.0 -0.330763  0.967133  1.132324  1.739077 -0.654336
4   -0.079546  0.0 -1.771079  1.879751  0.450286  1.804035  0.708613
..        ...  ...       ...       ...       ...       ...       ...
447 -0.079546  0.0  0.533427 -0.436895 -1.758926 -1.622502 -0.654336
448 -0.079546  0.0 -0.330763  0.826730  1.384382  1.782383  1.390087
449 -0.079546  0.0  2.837932 -0.156089 -4.709483 -3.392610 -0.427178
450 -0.079546  0.0 -0.042700  0.826730  0.420632  0.927101 -0.654336
451 -0.079546  0.0 -0.042700  0.265119  0.146334  0.180084 -0.654336

[452 rows x 280 columns]
```

In [ ]:

```python
# phương pháp MinMaxScaler
scaler = MinMaxScaler()
minmax = scaler.fit_transform(array)
minmax_df = pd.DataFrame(minmax, index = df.index)
print('Scaling dữ liệu:\n',minmax_df)
```

```
Scaling dữ liệu:
           0    1         2         3         4         5         6    \
0    0.903614  0.0  0.125926  0.435294  0.270677  0.368321  0.501805
1    0.674699  1.0  0.088889  0.341176  0.195489  0.332061  0.610108
2    0.650602  0.0  0.099259  0.523529  0.624060  0.311069  0.555957
3    0.662651  0.0  0.103704  0.517647  0.338346  0.385496  0.534296
4    0.903614  0.0  0.125926  0.435294  0.248120  0.345420  0.462094
..        ...  ...       ...       ...       ...       ...       ...
447  0.638554  1.0  0.081481  0.376471  0.187970  0.379771  0.541516
448  0.445783  0.0  0.125926  0.464706  0.338346  0.261450  0.465704
449  0.433735  0.0  0.090370  0.364706  0.398496  0.335878  0.480144
450  0.385542  1.0  0.074074  0.288235  0.285714  0.202290  0.555957
451  0.939759  1.0  0.081481  0.376471  0.180451  0.242366  0.476534

          7         8         9    ...       270       271       272  273  \
0    0.241758  0.590244  0.457478  ...  1.000000  0.381356  0.968531  0.0
1    0.150183  0.190244  0.577713  ...  1.000000  0.360169  1.000000  0.0
2    0.282051  0.497561  0.785924  ...  1.000000  0.402542  0.916084  0.0
3    0.260073  0.697561  0.586510  ...  1.000000  0.516949  0.923077  0.0
4    0.252747  0.502439  0.457478  ...  1.000000  0.555085  0.874126  0.0
..        ...       ...       ...  ...       ...       ...       ...  ...
447  0.168498  0.570732  0.395894  ...  1.000000  0.182203  0.825175  0.0
448  0.340659  0.356098  0.756598  ...  1.000000  0.661017  0.944056  0.0
449  0.315018  0.565854  0.255132  ...  1.000000  0.690678  0.000000  0.0
450  0.402930  0.307317  0.662757  ...  0.902439  0.508475  0.975524  0.0
451  0.109890  0.380488  0.586510  ...  1.000000  0.440678  0.937063  0.0

     274      275       276       277       278       279
0    0.0  0.53125  0.741667  0.507519  0.569579  0.466667
1    0.0  0.31250  0.675000  0.485714  0.500971  0.333333
2    0.0  0.34375  0.783333  0.424812  0.566990  0.600000
3    0.0  0.37500  0.716667  0.592481  0.648544  0.000000
4    0.0  0.21875  0.825000  0.523308  0.656311  0.400000
..   ...      ...       ...       ...       ...       ...
447  0.0  0.46875  0.550000  0.299248  0.246602  0.000000
448  0.0  0.37500  0.700000  0.618045  0.653722  0.600000
449  0.0  0.71875  0.583333  0.000000  0.034951  0.066667
450  0.0  0.40625  0.700000  0.520301  0.551456  0.000000
451  0.0  0.40625  0.633333  0.492481  0.462136  0.000000

[452 rows x 280 columns]
```

Rời rác hóa mỗi thuộc tính số hóa thành

- 10 equi-width ranges
- 10 equi-depth ranges

In [ ]:
```
equi_width_scaler = KBinsDiscretizer(
    n_bins=10, encode='ordinal', strategy='uniform')
equi_width_data = equi_width_scaler.fit_transform(df)
equi_width_data
```

```
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 19 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 67 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 69 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 83 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 131 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 132 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 139 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 141 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 143 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 145 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 151 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 156 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 157 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 164 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 204 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 264 is constant and will be replaced with 0.
  warnings.warn(
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-packages\sklearn\preprocessing\_
discretization.py:342: UserWarning: Feature 274 is constant and will be replaced with 0.
  warnings.warn(
```

Out[ ]:

```
array([[9., 0., 1., ..., 5., 5., 4.],
       [6., 9., 0., ..., 4., 5., 3.],
       [6., 0., 0., ..., 4., 5., 6.],
       ...,
       [4., 0., 0., ..., 0., 0., 0.],
       [3., 9., 0., ..., 5., 5., 0.],
       [9., 9., 0., ..., 4., 4., 0.]], shape=(452, 280))
```

Ta thấy có dòng báo lỗi như sau

```
c:\Users\lenovo\AppData\Local\Programs\Python\Python313\Lib\site-
packages\sklearn\preprocessing\_discretization.py:342: UserWarning: Feature 274 is
constant and will be replaced with 0.
  warnings.warn(f"Feature {i} is constant and will be replaced with 0.")
```

Vì vậy nên ta sẽ tiến hành loại bỏ các cột có toàn giá trị 0 trước khi rời rác hóa.

In [ ]:
```
zeros_cols = df.columns[(df == 0).all()]
df.drop(labels=zeros_cols, inplace=True, axis=1)

equi_width_scaler = KBinsDiscretizer(
    n_bins=10, encode='ordinal', strategy='uniform')
equi_width_data = equi_width_scaler.fit_transform(df)
equi_width_data
```

Out[ ]:
```
array([[9., 0., 1., ..., 5., 5., 4.],
       [6., 9., 0., ..., 4., 5., 3.],
       [6., 0., 0., ..., 4., 5., 6.],
       ...,
       [4., 0., 0., ..., 0., 0., 0.],
       [3., 9., 0., ..., 5., 5., 0.],
       [9., 9., 0., ..., 4., 4., 0.]], shape=(452, 263))
```

In [ ]:
```
equi_depth_scaler = KBinsDiscretizer(
    n_bins=10, encode='ordinal', strategy='quantile')
equi_depth_data = equi_width_scaler.fit_transform(df)
equi_depth_data
```

Out[ ]:
```
array([[9., 0., 1., ..., 5., 5., 4.],
       [6., 9., 0., ..., 4., 5., 3.],
       [6., 0., 0., ..., 4., 5., 6.],
       ...,
       [4., 0., 0., ..., 0., 0., 0.],
       [3., 9., 0., ..., 5., 5., 0.],
       [9., 9., 0., ..., 4., 4., 0.]], shape=(452, 263))
```

## 3. PCA

Ta thực hiện nạp dữ liệu

In [ ]:
```
df2 = pd.read_csv(r'C:\Users\lenovo\OneDrive\Document\Data-Mining_MTH10358\Homeworks\TH\1\musk+v
df2
```

Out[ ]:

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | ... | 159 | 160 | 161 | 162 | 163 | 16 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | MUSK-211 | 211_1+1 | 46 | -108 | -60 | -69 | -117 | 49 | 38 | -161 | ... | -308 | 52 | -7 | 39 | 126 | 15 |
| **1** | MUSK-211 | 211_1+10 | 41 | -188 | -145 | 22 | -117 | -6 | 57 | -171 | ... | -59 | -2 | 52 | 103 | 136 | 16 |
| **2** | MUSK-211 | 211_1+11 | 46 | -194 | -145 | 28 | -117 | 73 | 57 | -168 | ... | -134 | -154 | 57 | 143 | 142 | 16 |
| **3** | MUSK-211 | 211_1+12 | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | ... | -60 | -4 | 52 | 104 | 136 | 16 |
| **4** | MUSK-211 | 211_1+13 | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | ... | -60 | -4 | 52 | 104 | 137 | 16 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | |
| **6593** | NON-MUSK-jp13 | jp13_2+5 | 51 | -123 | -23 | -108 | -117 | 134 | -160 | 82 | ... | -66 | 164 | -14 | -29 | 107 | 17 |
| **6594** | NON-MUSK-jp13 | jp13_2+6 | 44 | -104 | -19 | -105 | -117 | 142 | -165 | 68 | ... | -51 | 166 | -9 | 150 | 129 | 15 |
| **6595** | NON-MUSK-jp13 | jp13_2+7 | 44 | -102 | -19 | -104 | -117 | 72 | -165 | 65 | ... | 90 | 117 | -8 | 150 | 130 | 15 |
| **6596** | NON-MUSK-jp13 | jp13_2+8 | 51 | -121 | -23 | -106 | -117 | 63 | -161 | 79 | ... | 86 | 99 | -14 | -31 | 106 | 17 |
| **6597** | NON-MUSK-jp13 | jp13_2+9 | 51 | -122 | -23 | -106 | -117 | 190 | -161 | 80 | ... | 40 | 124 | -14 | -30 | 107 | 17 |

6598 rows × 169 columns

Hiện thị thông tin tổng quan bộ dữ liệu

In [39]:
```
df2.info()
df2.describe()
print(df.dtypes)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6598 entries, 0 to 6597
Columns: 169 entries, 0 to 168
dtypes: float64(1), int64(166), object(2)
memory usage: 8.5+ MB
0        int64
1        int64
2        int64
3        int64
4        int64
        ...
275    float64
276    float64
277    float64
278    float64
279      int64
Length: 280, dtype: object
```

Xóa 2 cột đầu vì 2 cột đầu ta không có dùng đến

In [40]:
```
df2 = df2.iloc[:, 2:]
df2
```

Out[40]:

| | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | ... | 159 | 160 | 161 | 162 | 163 | 164 | 165 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 46 | -108 | -60 | -69 | -117 | 49 | 38 | -161 | -8 | 5 | ... | -308 | 52 | -7 | 39 | 126 | 156 | -50 |
| **1** | 41 | -188 | -145 | 22 | -117 | -6 | 57 | -171 | -39 | -100 | ... | -59 | -2 | 52 | 103 | 136 | 169 | -61 |
| **2** | 46 | -194 | -145 | 28 | -117 | 73 | 57 | -168 | -39 | -22 | ... | -134 | -154 | 57 | 143 | 142 | 165 | -67 |
| **3** | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | -39 | -99 | ... | -60 | -4 | 52 | 104 | 136 | 168 | -60 |
| **4** | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | -39 | -99 | ... | -60 | -4 | 52 | 104 | 137 | 168 | -60 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **6593** | 51 | -123 | -23 | -108 | -117 | 134 | -160 | 82 | -230 | -28 | ... | -66 | 164 | -14 | -29 | 107 | 171 | -44 |
| **6594** | 44 | -104 | -19 | -105 | -117 | 142 | -165 | 68 | -225 | -32 | ... | -51 | 166 | -9 | 150 | 129 | 158 | -66 |
| **6595** | 44 | -102 | -19 | -104 | -117 | 72 | -165 | 65 | -219 | -12 | ... | 90 | 117 | -8 | 150 | 130 | 159 | -66 |
| **6596** | 51 | -121 | -23 | -106 | -117 | 63 | -161 | 79 | -224 | -30 | ... | 86 | 99 | -14 | -31 | 106 | 171 | -44 |
| **6597** | 51 | -122 | -23 | -106 | -117 | 190 | -161 | 80 | -227 | -52 | ... | 40 | 124 | -14 | -30 | 107 | 171 | -44 |

6598 rows × 167 columns

Ta thực hiện kiểm tra xem dữ liệu có bị trùng hay không, nếu có thì ta sẽ loại bỏ các dòng trùng lặp đó đi

In [41]:
```
df2.isnull().sum().sort_values(ascending=False)
```

Out[41]:
```
2      0
3      0
4      0
5      0
6      0
      ..
164    0
165    0
166    0
167    0
168    0
Length: 167, dtype: int64
```

Tương tự, ta cũng sẽ loại bỏ các dòng có giá trị thiếu đi vì nếu để lại thì sẽ ảnh hưởng đến kết quả PCA

In [ ]:
```
((df2 == '?') == True).sum().sort_values(ascending=False)
```

Out[ ]:
```
2      0
3      0
4      0
5      0
6      0
      ..
164    0
165    0
166    0
167    0
168    0
Length: 167, dtype: int64
```

Tiếp theo ta lọc những dòng có giá trị không phải là số vì nếu để lại thì sẽ ảnh hưởng đến kết quả PCA

In [ ]:
```
df2_pca = df2.select_dtypes(exclude='object')

df2_pca = df2_pca[df2_pca.columns[:-1]]
df2_pca
```

Out[ ]:

| | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | ... | 158 | 159 | 160 | 161 | 162 | 163 | 164 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **0** | 46 | -108 | -60 | -69 | -117 | 49 | 38 | -161 | -8 | 5 | ... | -244 | -308 | 52 | -7 | 39 | 126 | 156 |
| **1** | 41 | -188 | -145 | 22 | -117 | -6 | 57 | -171 | -39 | -100 | ... | -235 | -59 | -2 | 52 | 103 | 136 | 169 |
| **2** | 46 | -194 | -145 | 28 | -117 | 73 | 57 | -168 | -39 | -22 | ... | -238 | -134 | -154 | 57 | 143 | 142 | 165 |
| **3** | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | -39 | -99 | ... | -236 | -60 | -4 | 52 | 104 | 136 | 168 |
| **4** | 41 | -188 | -145 | 22 | -117 | -7 | 57 | -170 | -39 | -99 | ... | -236 | -60 | -4 | 52 | 104 | 137 | 168 |
| **...** | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... | ... |
| **6593** | 51 | -123 | -23 | -108 | -117 | 134 | -160 | 82 | -230 | -28 | ... | 62 | -66 | 164 | -14 | -29 | 107 | 171 |
| **6594** | 44 | -104 | -19 | -105 | -117 | 142 | -165 | 68 | -225 | -32 | ... | 60 | -51 | 166 | -9 | 150 | 129 | 158 |
| **6595** | 44 | -102 | -19 | -104 | -117 | 72 | -165 | 65 | -219 | -12 | ... | -226 | 90 | 117 | -8 | 150 | 130 | 159 |
| **6596** | 51 | -121 | -23 | -106 | -117 | 63 | -161 | 79 | -224 | -30 | ... | -238 | 86 | 99 | -14 | -31 | 106 | 171 |
| **6597** | 51 | -122 | -23 | -106 | -117 | 190 | -161 | 80 | -227 | -52 | ... | 95 | 40 | 124 | -14 | -30 | 107 | 171 |

6598 rows × 166 columns

Thực hiện PCA để giảm chiều dữ liệu bằng cách sử dụng thư viện PCA của sklearn

In [54]:
```
pca = PCA()
pca_result = pca.fit_transform(df2_pca)
pca_result
```

Out[54]:
```
array([[-9.99463932e+01, -9.15817180e+02,  1.66686603e+02, ...,
        -2.00926382e+00,  6.49157635e-01,  9.22882593e-01],
       [ 2.08960040e+02, -8.28276632e+02,  7.17301732e+01, ...,
         8.50205472e-01, -6.72083911e-01,  8.79635949e-01],
       [ 2.50215211e+02, -7.95425387e+02, -2.01909466e+02, ...,
         2.67714147e-01,  1.28962297e+00,  1.16115622e+00],
       ...,
       [-9.70255709e+02,  1.37261091e+02, -3.67097607e+02, ...,
         5.39309269e+00,  8.16562866e-01,  1.65866567e+00],
       [-9.77148961e+02,  7.27506558e+01,  2.19302370e+02, ...,
         4.85167274e+00,  2.18189232e-01,  2.27511531e+00],
       [-8.98429502e+02,  1.66629988e+02,  2.08679827e+02, ...,
         3.16500000e-01,  5.76085154e-01,  7.32136474e-01]],
      shape=(6598, 166))
```

Ta thực hiện lấy giá trị eigenvalues và eigenvectors của ma trận covariance

In [53]:
```
eigenvectors = pca.components_
eigenvectors
```

Out[53]:
```
array([[-2.97137514e-03, -7.06755098e-02, -7.91938565e-02, ...,
        -1.41749460e-03, -1.22433553e-02,  3.08829012e-03],
       [-5.69905035e-02,  2.42059127e-02,  3.49982033e-02, ...,
        -3.62576743e-02, -6.48892199e-02,  2.32312750e-02],
       [-4.08873101e-03,  1.20755542e-01,  7.75935347e-03, ...,
         2.66274637e-02,  1.81740219e-02,  1.69540247e-01],
       ...,
       [ 5.77812789e-03, -1.01891098e-02,  8.55698836e-02, ...,
         6.55551196e-04,  1.29948160e-03,  2.40306043e-04],
       [-6.53473282e-05, -4.66232514e-02, -3.47297288e-01, ...,
        -5.53501371e-04,  2.01937322e-03,  1.89670467e-04],
       [-1.30070912e-03, -5.96284763e-03, -1.02747769e-01, ...,
         8.14628951e-04, -1.55090624e-03,  7.69406910e-04]],
      shape=(166, 166))
```

In [50]:
```
eigen_values = np.square(pca.singular_values_)
eigen_values
```

Out[50]:
```
array([3.00856869e+09, 1.70131869e+09, 5.00545894e+08, 3.03600798e+08,
       2.66840040e+08, 2.42452867e+08, 2.18125877e+08, 1.95551382e+08,
       1.67567178e+08, 1.59276140e+08, 1.27371946e+08, 1.18028904e+08,
       1.09542021e+08, 9.67449368e+07, 9.08952479e+07, 8.31696975e+07,
       7.75477812e+07, 7.09509848e+07, 6.10519026e+07, 5.96363685e+07,
       5.22180399e+07, 4.88041822e+07, 4.70140808e+07, 4.13277464e+07,
       3.62898059e+07, 3.59992110e+07, 3.52245456e+07, 3.30178651e+07,
       3.15269171e+07, 2.95502446e+07, 2.79135395e+07, 2.58060669e+07,
       2.47067783e+07, 2.29102832e+07, 2.24117704e+07, 2.09947458e+07,
       2.03357313e+07, 1.80377011e+07, 1.71562259e+07, 1.68244965e+07,
       1.52702182e+07, 1.46858833e+07, 1.37139738e+07, 1.27864116e+07,
       1.22622407e+07, 1.16869181e+07, 1.10112318e+07, 1.02234310e+07,
       9.55199051e+06, 9.03817751e+06, 8.72733735e+06, 8.31821227e+06,
       7.90212493e+06, 7.50435397e+06, 7.46891743e+06, 7.41877288e+06,
       6.80304920e+06, 6.45958360e+06, 6.28249246e+06, 5.74172143e+06,
       5.29841364e+06, 5.13264268e+06, 5.02927745e+06, 4.81362280e+06,
       4.70868194e+06, 4.60689048e+06, 4.38876274e+06, 4.20201682e+06,
       4.13119951e+06, 3.87602168e+06, 3.83399764e+06, 3.41335434e+06,
       3.28759077e+06, 3.15909146e+06, 3.02826059e+06, 2.82761131e+06,
       2.69246751e+06, 2.56598859e+06, 2.46726008e+06, 2.41216548e+06,
       2.25233982e+06, 2.09232941e+06, 2.01018705e+06, 1.97370546e+06,
       1.92386950e+06, 1.80870504e+06, 1.78068443e+06, 1.71572313e+06,
       1.59702340e+06, 1.55821754e+06, 1.53213317e+06, 1.45247595e+06,
       1.37930498e+06, 1.31483470e+06, 1.27773319e+06, 1.24835123e+06,
       1.21298306e+06, 1.18173189e+06, 1.14785227e+06, 1.06471434e+06,
       1.02806963e+06, 9.92836649e+05, 9.74209193e+05, 9.12100495e+05,
       8.84482002e+05, 8.66557715e+05, 7.93295826e+05, 7.69369775e+05,
       7.24247861e+05, 7.17221965e+05, 7.05426945e+05, 6.51494157e+05,
       6.36578031e+05, 6.22157666e+05, 5.93710373e+05, 5.90630943e+05,
       5.73114073e+05, 5.35703871e+05, 5.30975695e+05, 5.04307733e+05,
       4.67274250e+05, 4.50629497e+05, 4.45867625e+05, 4.19142420e+05,
       4.07808074e+05, 3.81590349e+05, 3.76201937e+05, 3.65268619e+05,
       3.41307849e+05, 3.23790024e+05, 3.17168611e+05, 3.14719430e+05,
       2.81452992e+05, 2.64400508e+05, 2.56454429e+05, 2.44309641e+05,
       2.28840936e+05, 2.26052325e+05, 2.05966289e+05, 1.96439703e+05,
       1.87964241e+05, 1.85282750e+05, 1.81524463e+05, 1.75656895e+05,
       1.56622449e+05, 1.48587130e+05, 1.38874812e+05, 1.27172678e+05,
       1.17593860e+05, 1.13812729e+05, 1.11432857e+05, 1.08393637e+05,
       1.00665992e+05, 8.88510826e+04, 8.57956983e+04, 7.72021333e+04,
       7.41393308e+04, 6.44994753e+04, 5.78131861e+04, 4.09322332e+04,
       3.95147601e+04, 3.87487045e+04, 3.28535162e+04, 2.79219446e+04,
       2.57807110e+04, 1.89884285e+04])
```